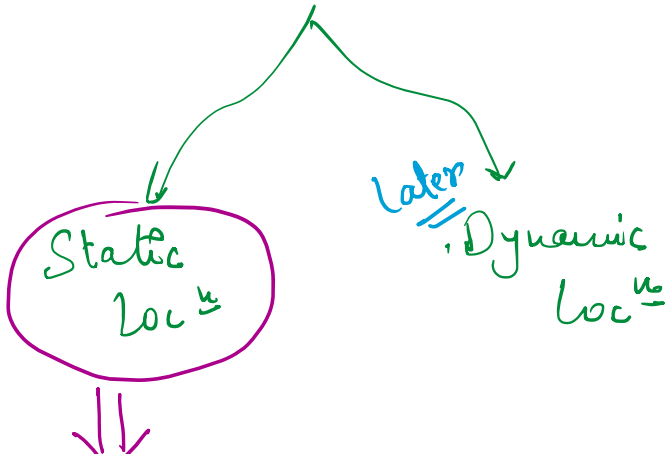
Case Study → YELP

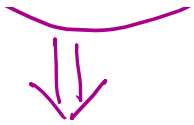
Overview

- A platform to promote your business
  - mostly on lines of JustDial.com

Build loc^n based systems

- Nearby Friends
- Swiggy
  - Nearby Restaurants
  - Find nearby Drivers
- Uber
  - Drivers near you
- Tinder, Bumble, Hinge - - -
  - Dates near you.

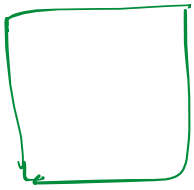




Case Study :- Finding restaurants near me.

API List<Restaurants> findRestaurantsNearby(Loc)

GET/restaurants
{lat:_, long:_}

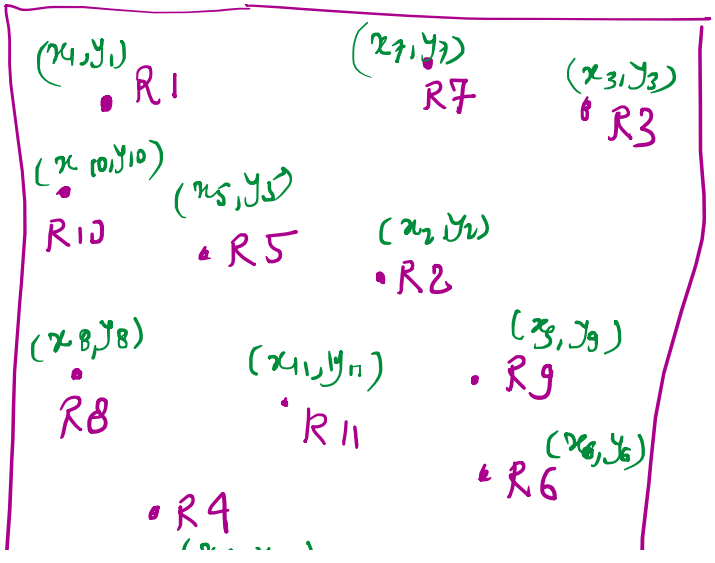


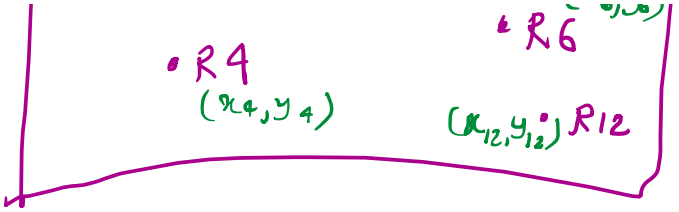
Server

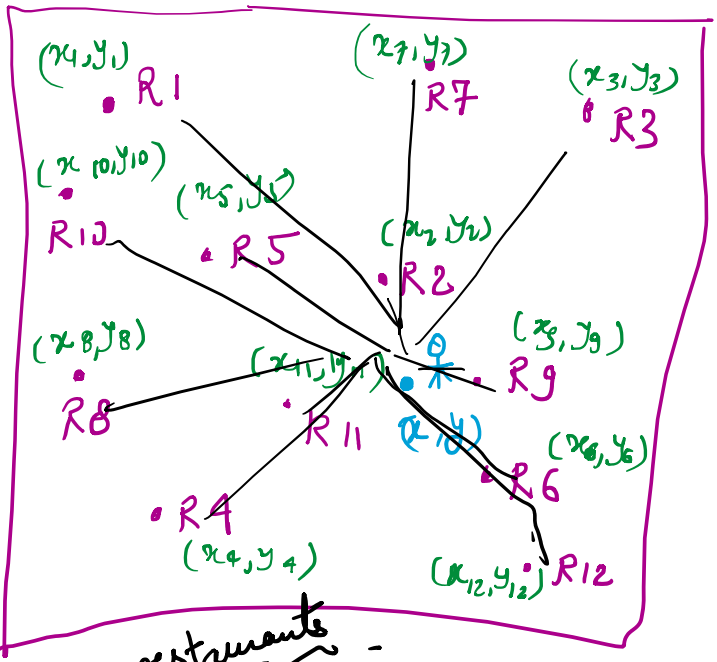
Brute Force

restaurants

id	title	rating	loc (lat, long.)







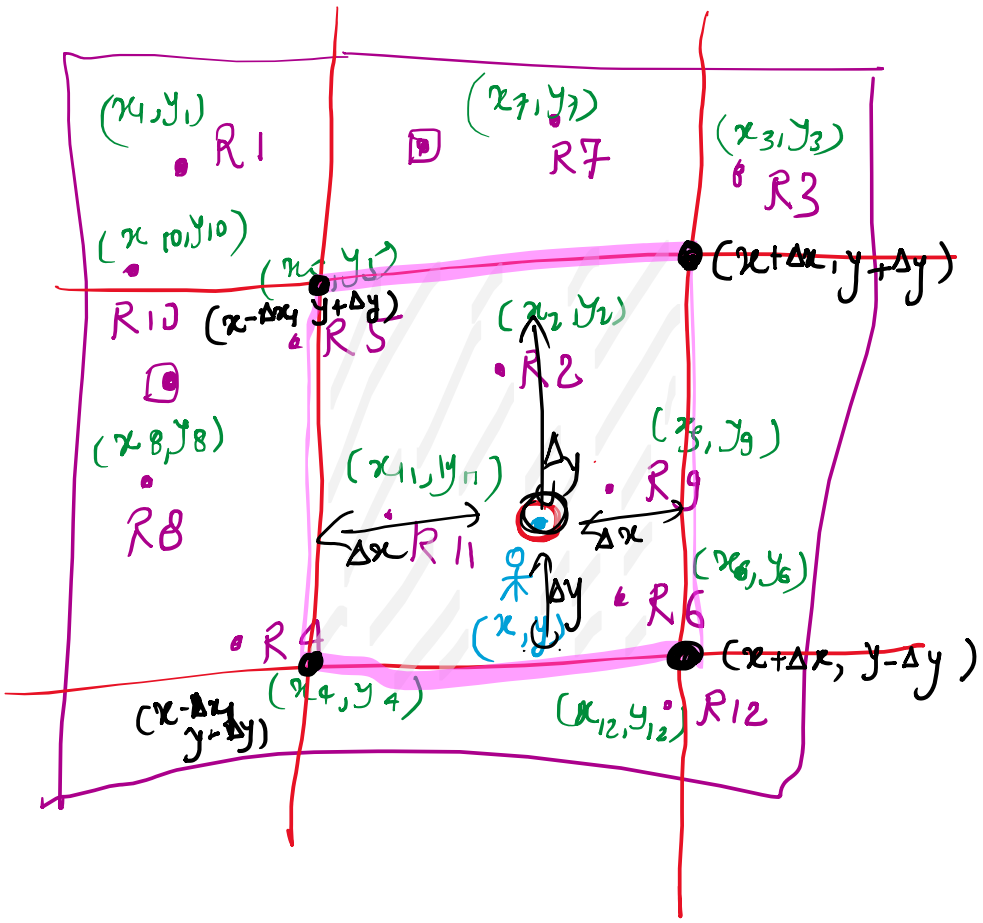
nearest no restaurants -

```
heap = [ ]
for every restaurant (x0, y0):
        dist = √((x0 - x)^2 + (y0 - y)^2)
        heap.put(dist, res);
        if (heap.size > 20)
                heap.pop()
```

vvvvvv slow → O(N)

N: no. of restaurants

Optimize #1



Instead of searching complete earth, try to find restaurants in a smaller area.

$\Delta x = x_0$

```
While (true) {
    Select *
    from restaurants
    where lat > x-Δx && lat < x+Δx
    and long > y-Δy && long < y+Δy

    If count > 20
```

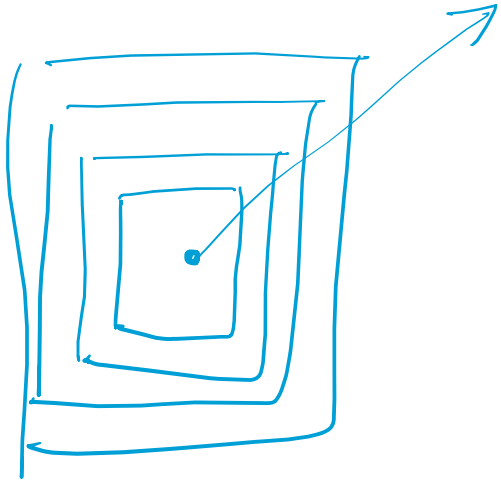
index

If count > 20

~~break~~; return

Δx += Δx ;

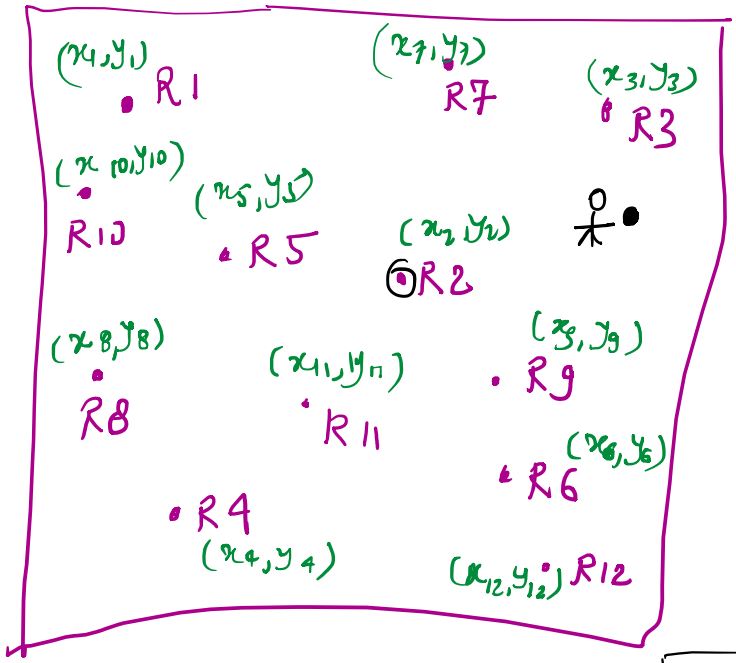
}



index

ON

Lat. & Long.



R3 - R7

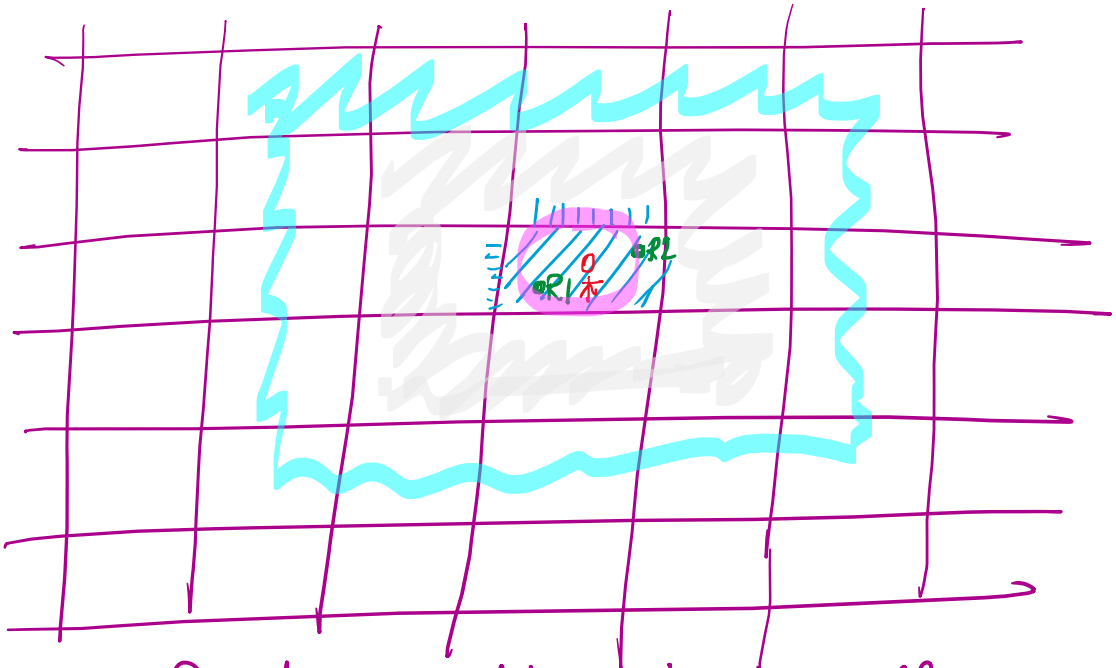
R7 - R2

HNSW -

Even w/ index,

cumbersome to implement.

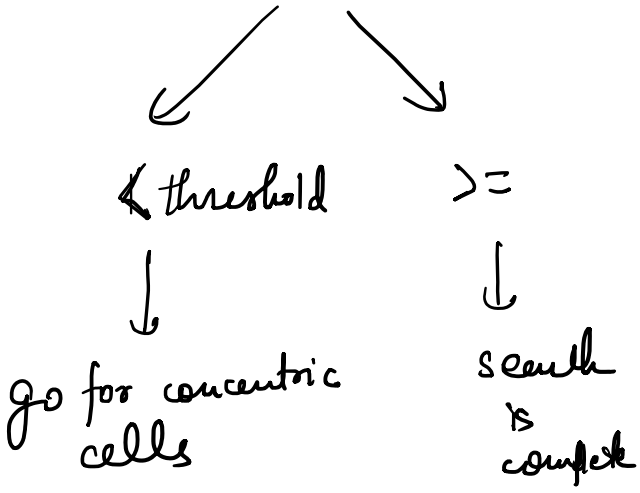
Optimize 2



Idea:- Divide world into small rectangles.

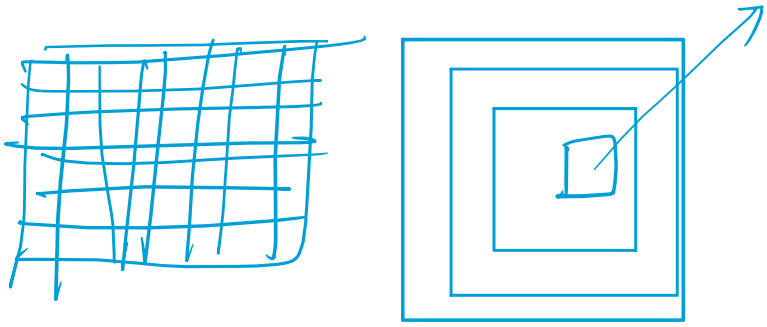
↳ given lat/long of user's loc^{n}
→ you know the cell id. → O(1)

↳ using that cell id
↓
find all restaurants in it



Q How will I divide the world into rectangles?

Ans
∴ Size of all rectangles is fixed,
We divide world into areas s.t.
each area has ~100 restaurants
threshold we want



But

Now!

the efficiency of req. will vary as per where the person is.

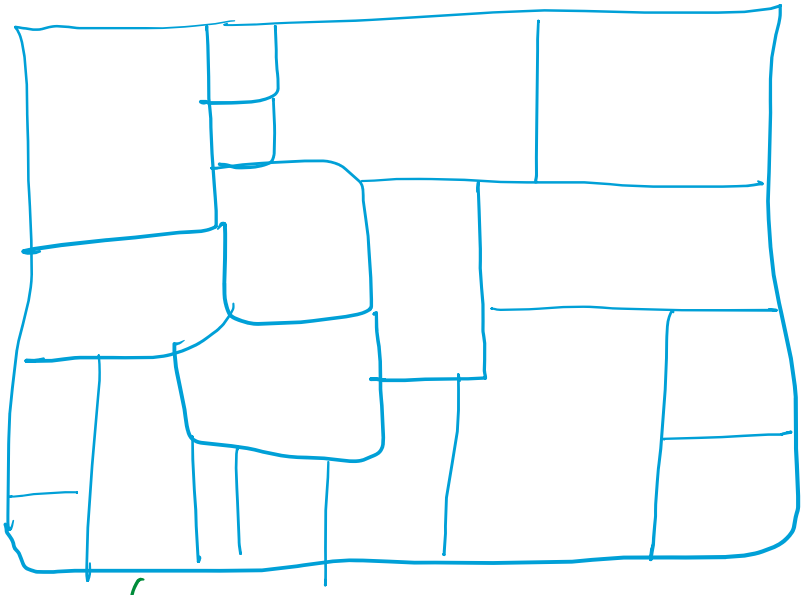
↓

HOW to further optimize ??

9:55 pm

↳ Sol^n

Variable-sized rectangles — Sl. 2. each rect. has 100 rectangles.

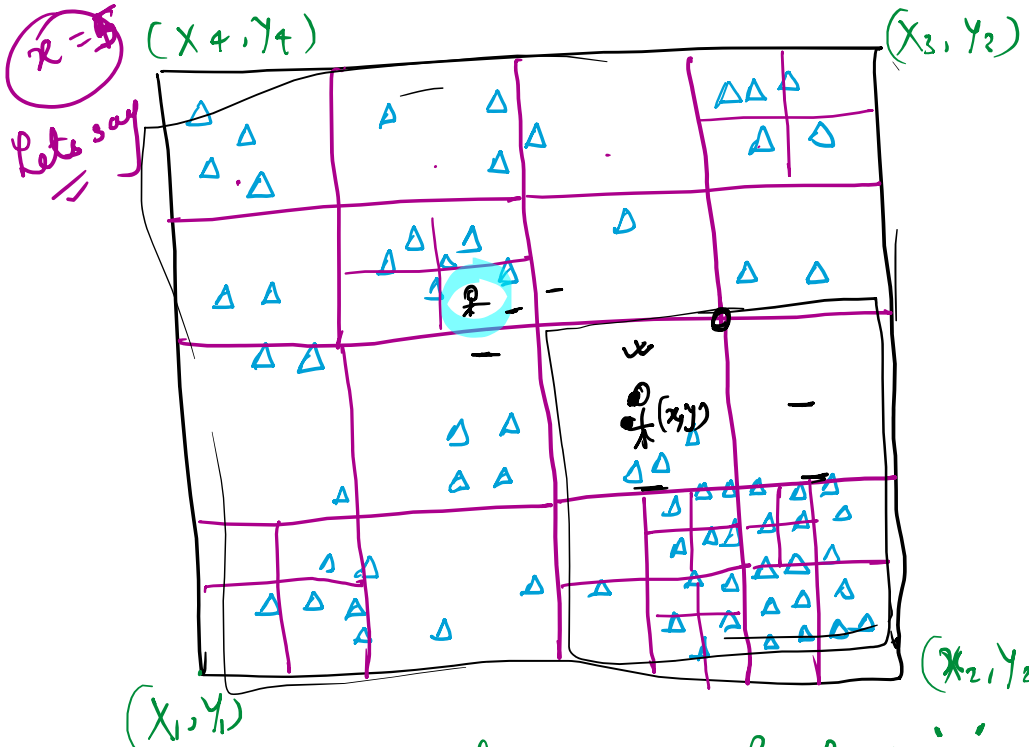


↓

HOW ?

Quad Trees

Quad Trees



Quad Tree →

Is the area having > 'x' no. of restaurants

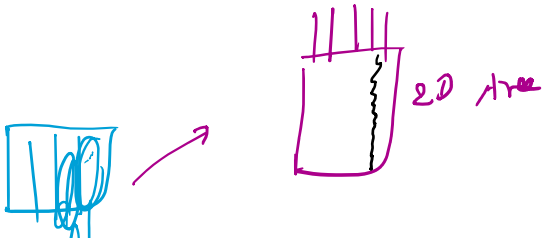
YES

NO

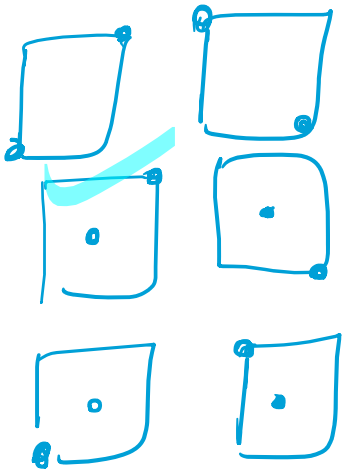
stop (no further divisions)

Divide that area into four equal parts

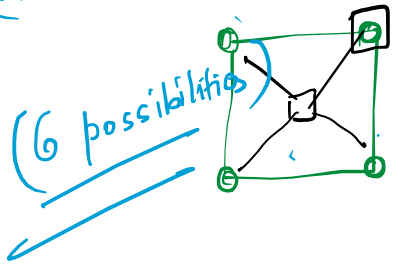
For all those newly formed areas

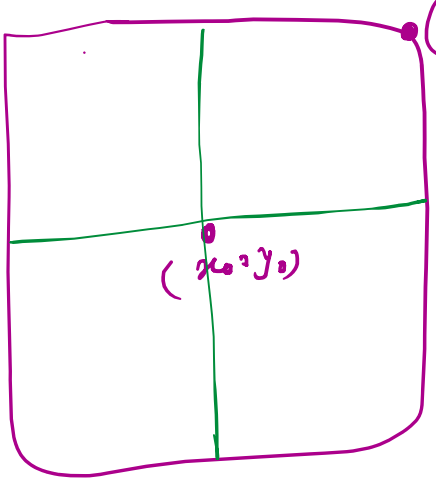


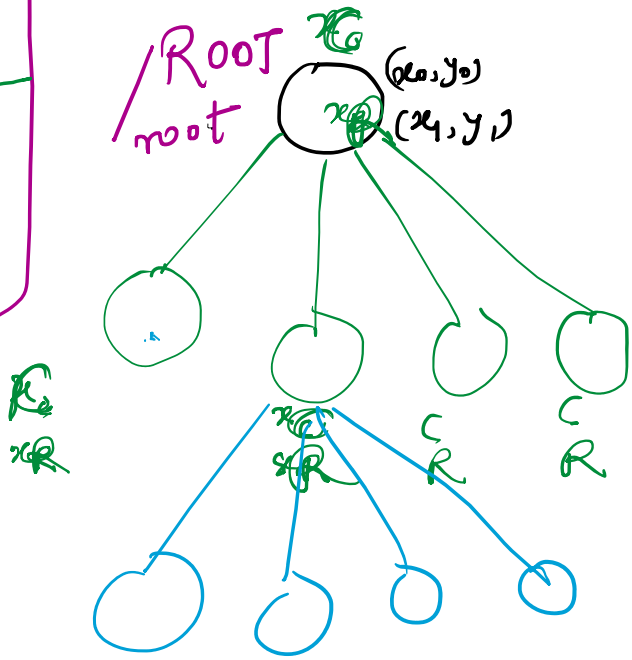




? ≡ how to represent a var-sized cell ? (in most efficient manner)







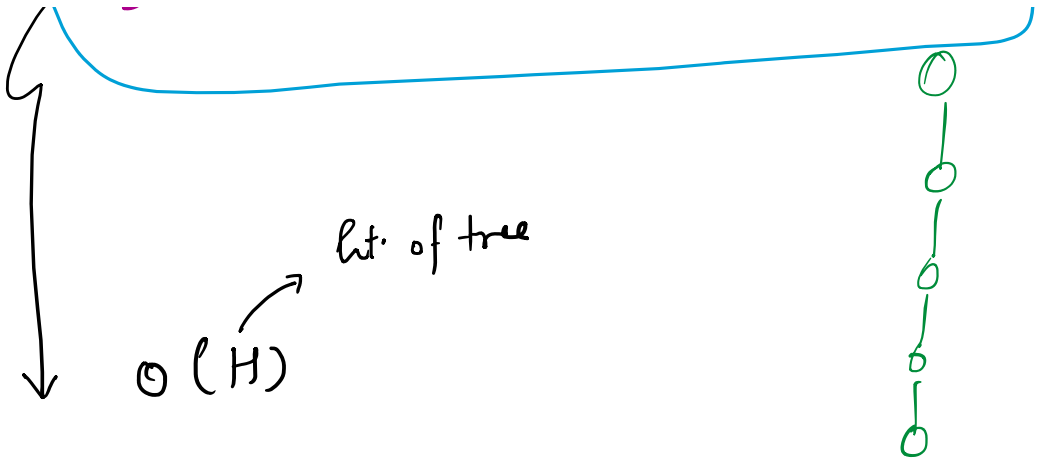
Q. What to maintain per node

- Center → (x_0, y_0)
- Top Right Corner → (x_1, y_1)
- no Of Restaurants (#)
  - 0 if # > 5
  - List if # <= 5
- List<Restaurants>
- List <Nodes> children → null → # <= 5
  - not null → # > 5 (4-members)

Q. Given a loc^{n}, how to go to the node for that loc^{n} ?

Ans

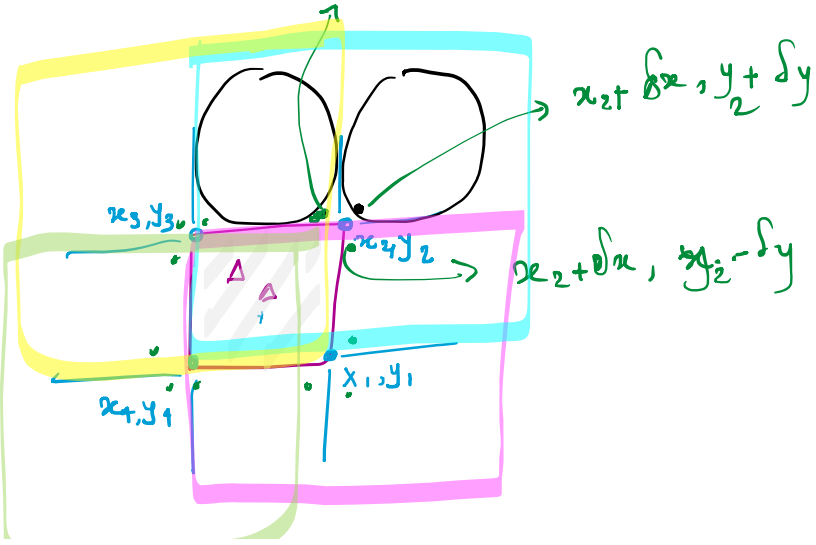
```
goToNode (locn) {
    ptr = root;
    while (ptr has children) {
        go to the child containing locn
    }
    return ptr;
}
```

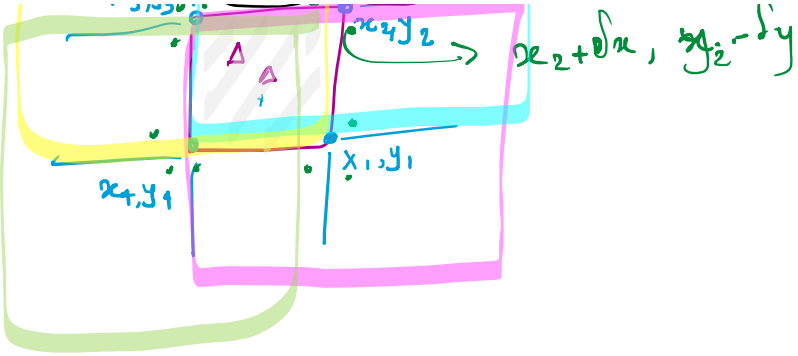



```
getRestaurantsNearMe (X, Y) {
    node = goToNode (X, Y);                                   → O(lg N)
    if (node.restaurants.size() >= 100) {                     → O(1)
        return node.restaurants;
    else :-
        List<Node> = nearbyLeaf (loc)                         → O(1)
                ↓ for each
        goToNode (diff; loc)                                  → 8 * O(lg N)
}
```

$x_2 - \delta x, x_2 + \delta y$

$O(\log N)$





↳ you only need to go for all nearby regions once
ie exactly 12 regions/times

(*) Next Class

↓

Build on top of above soln

for dynamic - loc^n based service

&

Storage of Nodes.